Expat NL Mortgage RAG

AI-Powered Property Advisory for International Buyers

RAG | Hybrid Search | Multi-Agent | Python | Qdrant | Streamlit

Ajai | March 2026 | 4,500+ Lines of Python

Presentation Roadmap

Covering all 8 evaluation criteria in depth

1. **Problem Definition & AI Framing**
   Why AI? Inputs, outputs, success criteria

2. **Data Understanding & Features**
   Dataset, preprocessing, semantic chunking

3. **AI Innovation & Insights**
   Hybrid RAG, multi-agent, knowledge graph

4. **Model Selection & Reasoning**
   Why RAG over fine-tuning? Provider abstraction

5. **Training, Evaluation & Metrics**
   RAGAS evals, golden dataset, drift monitoring

6. **Database-AI Workflow**
   Qdrant pipeline, end-to-end ingestion flow

7. **Business Impact**
   Who benefits? Actionable outputs

8. **Documentation & Reproducibility**
   Project structure, deployment, responsible AI

CRITERION 1

Problem Definition & AI Framing

The Problem

Language Barrier

Dutch mortgage docs are complex, in Dutch, with domain-specific terminology.

Information Overload

Tax rules (hypotheekrenteaftrek), NHG, woning types, energielabel — scattered across 100s of pages.

Time Pressure

Expats need fast, accurate answers when making one of their biggest financial decisions.

No Personalization

Generic websites don't answer: "Can I deduct mortgage interest as a kennismigrant?"

Why AI, Not Rules?

Unstructured Data

Mortgage documents are free-text PDFs — rules can't parse meaning from paragraphs.

Nuanced Queries

"What tax benefits apply to me as a 30% ruling holder?" requires contextual reasoning.

Evolving Knowledge

Tax laws and regulations change; AI retrieves from current documents, not hardcoded logic.

Multi-Domain

Combines mortgage, tax, location, and property data — too complex for if-else trees.

CRITERION 1

AI Task Definition

INPUTS

PDF mortgage documents

User natural-language query

Location/address data

Mortgage parameters
(bid, eigen inleg, type woning)

Web search results (Tavily)

AI PROCESSING

Semantic chunking
(800 char, structure-aware)

Vector embedding
(text-embedding-3-small)

Hybrid retrieval
(Vector + Keyword + RRF)

LLM reasoning
(GPT-4o / Claude / Ollama)

Multi-agent routing
(retrieval, location, calc)

OUTPUTS

Cited answer with sources

Mortgage cost estimates

Nearby facility map

Knowledge graph
visualization

Sun orientation diagram

Success Criteria: Faithfulness > 0.75 | Answer Relevancy > 0.90 | Response < 5s | Cited sources for every answer

CRITERION 2

Data Understanding

PDF Documents

Dutch mortgage guides, tax regulations,
property advisory documents.
Free-text, tables, legal terminology.

Unstructured

User Queries

Natural language questions from expats.
"Can I deduct mortgage interest?"
"What is NHG?"

Text Input

Location Data

Addresses, GPS coordinates, POI data
from OpenStreetMap (Nominatim, Overpass).
Route data from OSRM.

Geospatial

Golden Dataset

3 curated QA pairs with reference answers
and contexts for RAGAS evaluation.
Faithfulness & relevancy scored.

Evaluation

Data Challenges & How We Handle Them

Unstructured PDFs
pypdf extraction + semantic chunking preserves paragraph structure and meaning

Noisy text
Structure-aware splitting on paragraph breaks, markdown headers, numbered sections

Missing context
Hybrid retrieval (vector + keyword) + optional Tavily web search fallback

Domain terminology
800-char chunks with 150-char overlap ensure Dutch mortgage terms stay in context

CRITERION 2

Feature Engineering & Preprocessing

How raw PDFs become queryable knowledge — the semantic chunking pipeline

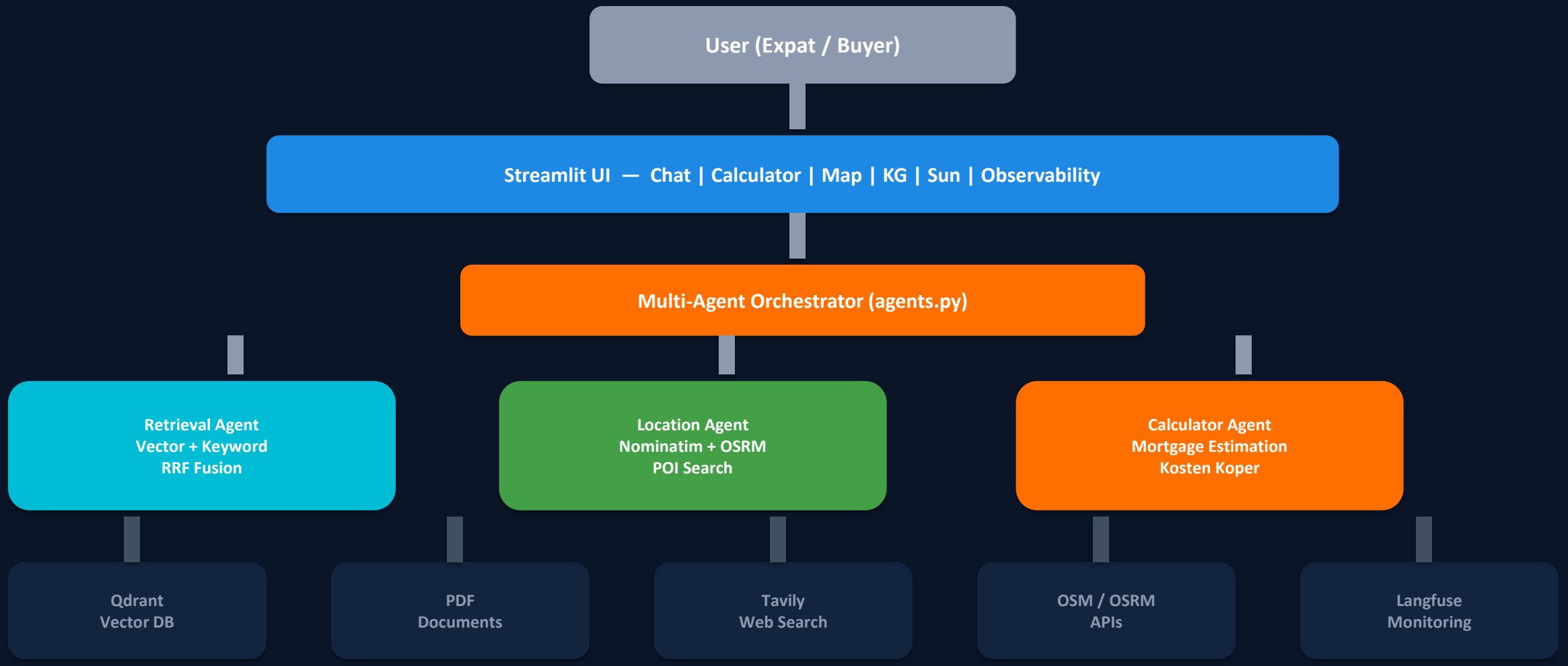
Key Parameters

Chunk Size	`800 chars`	Balances context richness with retrieval precision	**Overlap**	`150 chars`	Prevents splitting Dutch compound terms mid-word
Max Chunk	`2,000 chars`	Triggers LLM-based splitting for long sections	**Min Chunk**	`100 chars`	Filters out fragments too short to be useful
Embedding	`text-embedding-3-small`	1536 dimensions, OpenAI, cost-efficient	**Batch Size**	`100 chunks`	Optimizes API throughput during ingestion
Distance	`Cosine`	Standard for semantic similarity in RAG systems	**Vector DB**	`Qdrant`	Purpose-built for vector search with filtering

CRITERION 3

System Architecture Overview

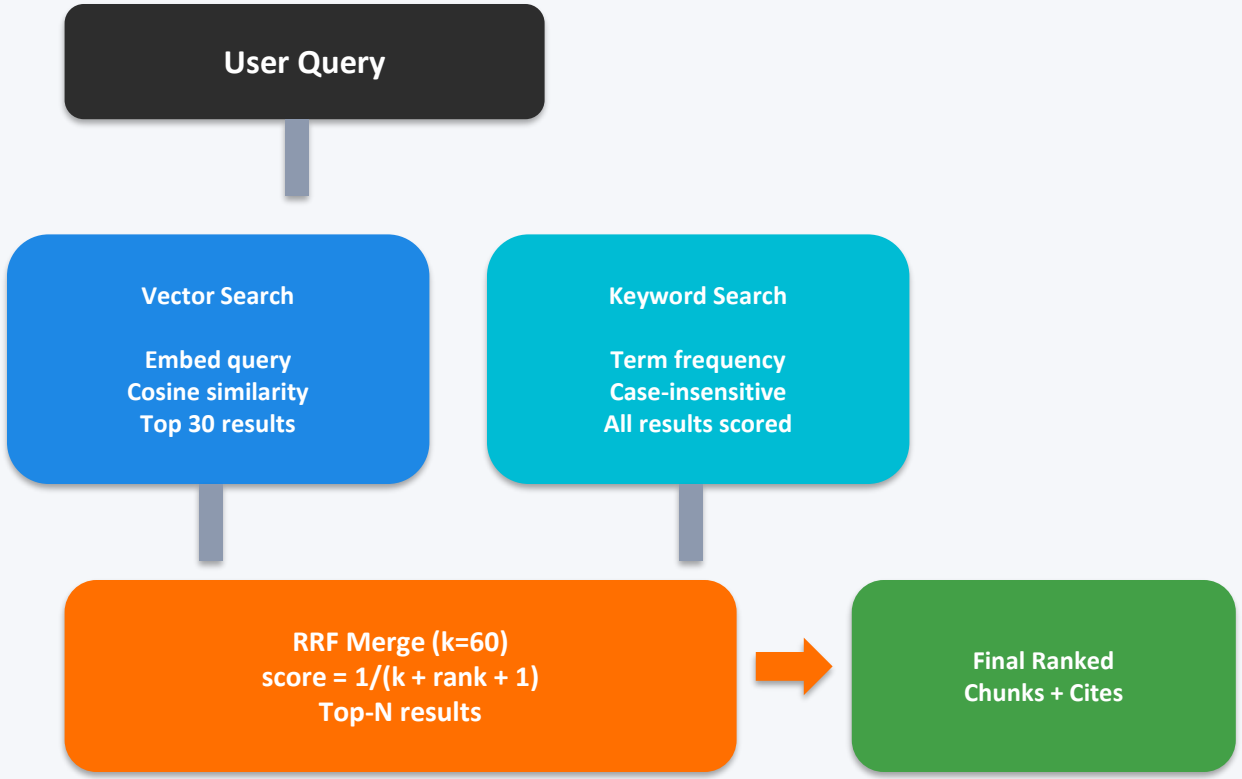
End-to-end RAG system with multi-agent orchestration and rich UI



CRITERION 3

AI Innovation: Hybrid Retrieval with RRF

Combining semantic understanding with keyword precision for better recall



Why Hybrid Search?

1. **Vector-only misses keywords**

   "NHG" might not be semantically close to "Nationale Hypotheek Garantie" in embedding space.

2. **Keyword-only misses meaning**

   "Can I deduct housing costs?" won't match "hypotheekrenteaftrek" by keyword alone.

3. **RRF fusion is robust**

   Reciprocal Rank Fusion doesn't need score normalization — works across different scoring scales.

4. **Configurable k parameter**

   k=60 balances early vs. late ranks. Higher k gives more weight to lower-ranked results.

CRITERION 3

AI Innovation: Multi-Agent Orchestration

Specialist agents collaborate to answer complex multi-domain queries

Query Router / Orchestrator

Retrieval Agent

Handles document-based queries.
Searches Qdrant with hybrid retrieval.
Returns cited chunks with sources.

Triggers: tax, deduction, mortgage rules, NHG, hypotheek terms

Location Agent

Handles spatial queries.
Geocodes via Nominatim + Overpass.
Computes OSRM commute routes.

Triggers: near, address, commute, school, supermarket, safety

Calculator Agent

Handles mortgage calculations.
Estimates bruto maandlasten, hypotheek, kosten koper.

Triggers: monthly, calculate, eigen inleg, maandlast

Agent-to-UI (A2UI) Directives

`show_map`

Location agent triggers map tab with POI markers and route overlays

`show_calculator`

Calculator agent opens mortgage calculator with pre-filled parameters

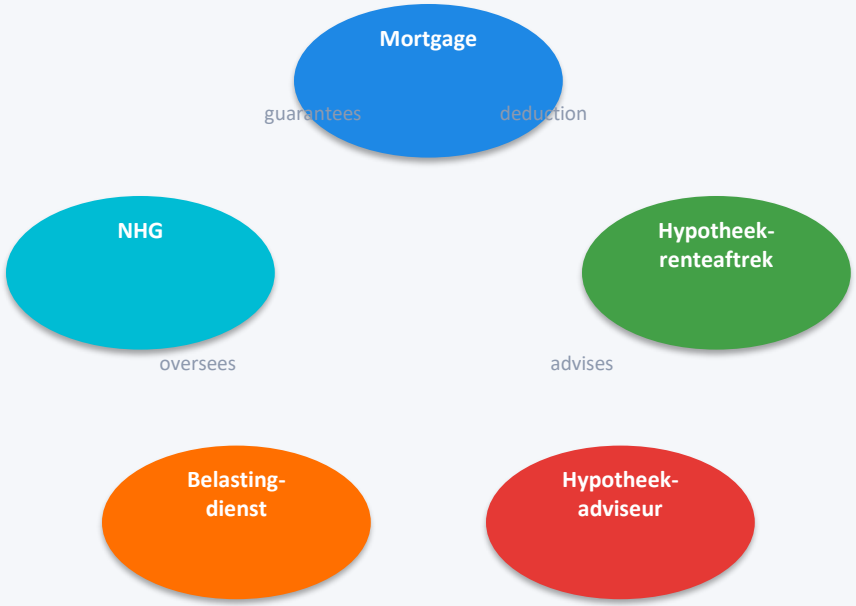
`show_citations`

Retrieval agent renders expandable source panel with document excerpts

CRITERION 3

Knowledge Graph & Location Tools

Knowledge Graph Extraction



Entities & relationships extracted from mortgage documents. Visualized with PyVis network graphs. Interactive, zoomable, and helps users understand domain connections.

Location Intelligence Tools

Nearby POI Search

10 categories: schools, grocery, hospitals, gyms, parks, restaurants, banks, transit...
Radius-based with Overpass API

Commute Routes

OSRM routing for walk/bike/car profiles.
Duration and distance calculations.
Origin-to-destination optimization

Area Safety

CBS/data.overheid.nl integration (placeholder).
Wijk/buurt level safety scoring.
Crime and livability indicators

Sun Orientation

Solar position calculations for properties.
Date slider + orientation control.
Interactive SVG visualization

CRITERION 4

Model Selection & Technical Reasoning

Why RAG, Not Fine-Tuning?

RAG (Retrieval-Augmented Generation)

- Documents can be updated without retraining
- Answers are grounded in real sources (citations)
- Works with any base LLM (swap models freely)
- Cost-effective: no GPU hours for training
- Transparent: users see which documents were used

Fine-Tuning (Not Chosen)

- Requires curated training dataset
- Can hallucinate without source attribution
- Locked to specific model version
- Expensive compute for each update cycle
- "Black box" — hard to explain answers

Flexible Provider Abstraction

OpenAI

`gpt-4o-mini, gpt-4o,`
`gpt-4-turbo, gpt-3.5-turbo`

Best quality, API-based

OpenRouter

`claude-3.5-sonnet,`
`gemini-2.0-flash, nemotron`

Multi-model gateway

Ollama

`llama3.2:3b, llama3.1:8b,`
`qwen2.5:7b-instruct`

Local, privacy-first, free

CRITERION 5

Training, Evaluation & Metrics

RAGAS evaluation framework with golden dataset and continuous monitoring

RAGAS Evaluation Results

0.77

Faithfulness (mean)

1.00

Answer Relevancy

Golden Dataset (3 QA Pairs)

Q: Can I deduct mortgage interest?

Tests: tax knowledge, hypotheekrenteaftrek

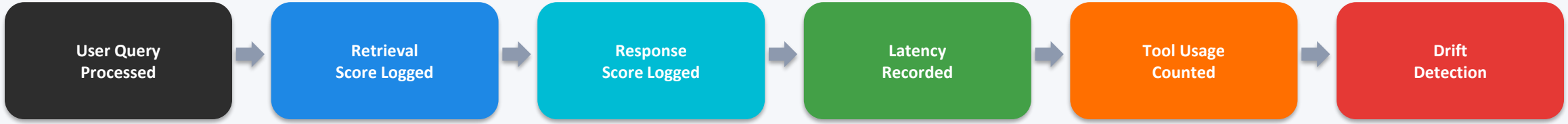
Q: What is NHG?

Tests: financial product knowledge

Q: Who helps expats with mortgages?

Tests: service provider retrieval

Continuous Monitoring Pipeline



Tracked: retrieval_scores (last 100) | response_scores (last 100) | request_latencies_ms (last 200) | tool_usage_counts | drift trend (up/stable/down)

CRITERION 5

Model Performance & Limitations

Understanding what the model does well and where it needs improvement

Strengths

- High answer relevancy (1.0) — responses directly address the query
- Cited sources — users can verify every claim against original documents
- Hybrid retrieval catches both semantic and keyword matches
- Multi-provider fallback — if OpenAI is down, switch to OpenRouter/Ollama
- Streaming responses — users see answers build in real-time

Known Limitations

- Faithfulness at 0.77 — room for improvement via prompt tuning
- Golden dataset is small (3 pairs) — needs expansion for robustness
- No A/B testing framework for prompt variants
- Drift detection is naive (mean comparison, not statistical)
- No user feedback loop yet (thumbs up/down not wired)

Improvement Plan

1. **Expand golden dataset** — Target: 50+ QA pairs covering all mortgage topics and edge cases
2. **Wire Langfuse tracing** — Track per-request faithfulness, latency, and token usage in production
3. **Add user feedback** — Thumbs up/down on each response feeds back into retrieval quality scoring
4. **Statistical drift tests** — Replace mean comparison with t-test and CUSUM for early anomaly detection

CRITERION 6

Database-AI Workflow

End-to-end data flow from PDF ingestion through AI processing to stored results

INGESTION PIPELINE

PDF Upload → pypdf Extract → Semantic Chunk → OpenAI Embed → Qdrant Upsert

QUERY PIPELINE

User Query → Embed Query → Hybrid Search → LLM Reasoning → Cited Answer

Data Persistence Layer

Qdrant Vector DB

Chunks + embeddings (1536-dim)
Cosine distance, batch upsert
Source metadata for citations

JSON Metrics Store

rag_metrics.json persistence
Retrieval/response scores
Latencies and tool usage

Golden Dataset

golden_rag.json (3 QA pairs)
ragas_scores.json (eval results)
Reference answers + contexts

Session State

Streamlit session for chat history
Provider/model selection
UI preferences persistence

CRITERION 6

Qdrant Vector Database Integration

Document Storage Schema

```
Collection: property_docs

Point Structure:
  id:      UUID (auto-generated)
  vector:  float[1536]  (cosine)
  payload:
    text:   str   (chunk content)
    source: str   (PDF filename)

Operations:
  create_collection()  # if not exists
  delete(source=...)   # remove old chunks
  upsert(batch=100)    # batch embedding
  search(vector, limit) # cosine NN
```

Search Flow

1. Embed query

text-embedding-3-small converts query to 1536-dim vector

2. Vector search

Qdrant returns top-30 nearest neighbors by cosine similarity

3. Keyword score

Each result scored by query term frequency in chunk text

4. RRF merge

Reciprocal Rank Fusion combines both rankings (k=60)

5. Top-N select

Final top-N chunks returned (configurable 3-20, default 10)

6. Build context

Chunks formatted with source citations for LLM prompt

7. Store metrics

Retrieval score + latency recorded for monitoring

CRITERION 7

Business Impact & Practical Use

Turning AI capabilities into actionable decisions for real users

1

Expat Home Buyers

Get instant, cited answers about Dutch mortgage rules, tax deductions, and NHG eligibility — in English.

2

Mortgage Advisors

Pre-screen client questions, quickly look up regulations, reduce time per consultation.

3

Real Estate Agents

Help international clients understand buying costs (kosten koper), property analysis, and nearby amenities.

Actionable Outputs

Cited Q&A Responses

Every answer links back to the specific document and paragraph. Users verify claims before making decisions.

Mortgage Cost Estimates

Quick calculation of bruto maandlasten, hypotheek amount, and kosten koper. Helps set budget expectations.

Location Analysis

Map of nearby schools, grocery stores, hospitals, transit. Commute time calculations. Area comparison.

Knowledge Visualization

Interactive knowledge graph shows how mortgage concepts connect. Helps newcomers learn the Dutch system.

CRITERION 7

Real-World Use Case Scenarios

Scenario 1: Tax Deduction Query

"Can I deduct mortgage interest as a 30% ruling holder?"

RAG retrieves hypotheekrenteaftrek docs + 30% ruling specifics.
LLM generates cited answer explaining eligibility and conditions.
User sees exact source paragraphs for verification.

Scenario 2: Property Comparison

"Compare schools and commute near Amstelveen vs. Haarlem"

Location agent geocodes both areas, queries Overpass for schools.
OSRM calculates commute to Amsterdam Centraal from each.
Map renders POI markers with distance/duration overlays.

Scenario 3: Budget Planning

"What can I afford with 80K salary and 50K savings?"

Calculator estimates hypotheek based on income multiples.
Shows kosten koper (~6% of bid), eigen inleg allocation.
Retrieval adds context about NHG limits and energielabel impact.

Scenario 4: First-Time Buyer Guide

"Explain the Dutch mortgage process for an expat"

Retrieval pulls comprehensive process docs.
Knowledge graph shows entity relationships (NHG, adviseur, etc.).
A2UI renders both citations and KG tab for deep exploration.

CRITERION 7

Streamlit UI: 9 Interactive Tabs

Rich user interface with real-time AI responses and interactive visualizations

Chat

RAG conversation with streaming responses, tools used display, and expandable source citations

Calculator

ING-style mortgage estimation. Bid, eigen inleg, woning type, energielabel dropdown

Map

Nearby facilities by category (walk/bike/car). PyDeck or Folium interactive map

Documents

PDF upload with text extraction. Optional KG extraction on ingestion. Document list view

Knowledge Graph

PyVis network visualization. Entities and relationships from mortgage domain. Interactive zoom

Location

Nearby POI search (10 categories). OSRM commute calculation. Area safety information

Sun Orientation

Solar position for properties. Date slider + orientation control. Interactive SVG diagram

Observability

Langfuse link, token tracking, retrieval/response quality, drift indicators dashboard

Agents (P4)

MCP tools registry display. Phase 4 orchestrator config. Specialist agent status

CRITERION 8

Documentation & Explainability

Clear project structure, comprehensive docs, and responsible AI practices

Project Structure

```
app.py                    Main entry point (Phase 1-4)
lib/
  retrieval.py            Hybrid search + RRF
  chunking.py             Semantic chunking
  documents.py            PDF extraction + Qdrant
  provider.py             LLM provider abstraction
  agents.py               Multi-agent orchestrator
  graph_kg.py             Knowledge graph extraction
  location.py             Nominatim + OSRM tools
  sun_orientation.py      Solar position calculations
  a2ui.py                 Agent-to-UI directives
  mcp_client.py           MCP tool registry
monitoring/               Drift detection + metrics
scripts/                  Ingestion, testing, evals
tests/                    pytest suite
data/                     Golden dataset + RAGAS scores
```

Documentation Artifacts

PHASES.md — 690 lines

Comprehensive 4-phase roadmap with features, architecture, and milestones

DEPLOYMENT.md — Detailed

Deployment guides for Streamlit Cloud, Hugging Face Spaces, and Render

README.md — Quick start

Setup instructions, project overview, and module descriptions

.env.example — 30+ vars

Complete configuration template with all required and optional variables

RESPONSIBLE_AI.md — Policy

Transparency, fairness, and limitations disclosure for AI-generated advice

CI/CD Pipeline — .github/

Automated linting (ruff) and testing (pytest) on every push/PR

CRITERION 8

Reproducibility & Getting Started

5-Step Quick Start

1. **Clone & Configure**

```
cp .env.example .env
Set OPENAI_API_KEY and QDRANT_URL
```

2. **Install Dependencies**

```
pip install -r requirements.txt
(48 packages, all pinned versions)
```

3. **Start Qdrant**

```
docker run -p 6333:6333 qdrant/qdrant
(or use Qdrant Cloud hosted)
```

4. **Ingest Documents**

```
python scripts/ingest_docs.py
(PDFs → chunks → embeddings → Qdrant)
```

5. **Launch Application**

```
streamlit run app.py
(opens at http://localhost:8501)
```

Deployment Platforms

Local Development	Docker + Streamlit Full control, all features
Streamlit Cloud	Connect GitHub repo Secrets via dashboard
Hugging Face Spaces	Streamlit Space type Hosted Qdrant backend
Render	Web service deployment Custom port and env vars

CRITERION 8

Responsible AI Practices

Transparency, fairness, and ethical considerations in AI-generated financial advice

1 Transparency

Every answer includes source citations.
Users can verify claims against original documents.
Model limitations are clearly communicated.

Users never take AI advice blindly — they see the evidence.

2 Accuracy Bounds

RAGAS evaluation measures faithfulness.
Golden dataset provides quality baseline.
Drift monitoring alerts on degradation.

The system knows when it doesn't know — and says so.

3 Privacy

Ollama option for fully local inference.
No user data sent to third parties (with local setup).
API keys are env-only, never logged.

Users control where their data goes.

4 Financial Disclaimer

Calculator outputs are clearly marked as estimates.
System explicitly states it is not a financial advisor.
Always recommends consulting a hypotheekadviseur.

AI augments, never replaces, professional advice.

4-Phase Evolution Roadmap

From foundation MVP to production multi-agent system

Phase 1
Foundation MVP

- RAG chat with citations
- Hybrid retrieval (RRF)
- Mortgage calculator
- Multi-provider LLM
- Web search (Tavily)
- Observability tab

COMPLETE

Phase 2
Location Intelligence

- Map + nearby POIs
- OSRM commute routing
- Area safety (CBS)
- Knowledge graph (PyVis)
- A2UI directives

IN PROGRESS

Phase 3
Monitoring & UX

- Sun orientation SVG
- RAGAS eval pipeline
- Drift detection
- Prometheus metrics
- Responsible AI docs

PLANNED

Phase 4
Multi-Agent

- Orchestrator routing
- Specialist agents
- A2A handoffs
- MCP integration
- LangGraph (optional)

PLANNED

Technology Stack

48 dependencies organized by purpose — all open-source or free-tier compatible

Frontend & UI

Streamlit	Web app framework
PyDeck/Folium	Map visualization
PyVis	Knowledge graph viz
NetworkX	Graph operations

LLM & AI

OpenAI SDK	GPT-4o, embeddings
LangChain	Agent framework
Ollama	Local LLM inference
Tavily	Web search API

Vector Store

Qdrant Client	Vector DB operations
Sentence Transformers	Embedding models
FastEmbed	Fast local embeddings
pypdf	PDF text extraction

Location

Nominatim (OSM)	Geocoding
Overpass API	POI queries
OSRM	Route calculation
Neo4j	Graph database (P2)

Monitoring

Prometheus Client	Metrics export
FastAPI/Uvicorn	Metrics server
Langfuse	LLM observability
JSON Store	Drift tracking

DevOps

GitHub Actions	CI/CD pipeline
Ruff	Linting
pytest	Testing
Docker	Containerization

Self-Assessment Against Criteria

How our project maps to the 8 evaluation dimensions

Strong	**1. Problem Definition & AI Framing**	Clear objective (RAG for mortgage Q&A), defined inputs/outputs, realistic scope, AI justified over rules
Strong	**2. Data Understanding & Features**	PDF preprocessing, semantic chunking (800/150), hybrid search features, domain-aware splitting
Strong	**3. AI Innovation & Insights**	Hybrid RRF retrieval, multi-agent orchestration, KG extraction, location intelligence, A2UI directives
Strong	**4. Model Selection & Reasoning**	RAG chosen over fine-tuning with clear rationale, flexible provider abstraction, model limitations stated
Good	**5. Training, Evaluation & Metrics**	RAGAS framework, golden dataset, faithfulness 0.77, relevancy 1.0, continuous monitoring pipeline
Strong	**6. Database-AI Workflow**	Qdrant vector DB, end-to-end ingestion, batch embedding, source tracking, metrics persistence
Strong	**7. Business Impact**	Clear beneficiaries (expats, advisors), actionable outputs with citations, 4 real-world scenarios
Strong	**8. Documentation & Explainability**	690-line PHASES.md, DEPLOYMENT.md, README, responsible AI policy, CI/CD pipeline, .env.example

Summary & Key Takeaways

What We Built

- RAG system for Dutch mortgage advice
- Hybrid retrieval with RRF fusion
- Multi-agent specialist orchestration
- 9-tab Streamlit interface
- 4,500+ lines of Python

How It Innovates

- Combines vector + keyword search
- Agents route to specialists automatically
- Knowledge graph for domain concepts
- Location intelligence with commute data
- A2UI for dynamic widget rendering

Real Impact

- Expats get cited, verifiable answers
- Mortgage advisors save consultation time
- Property comparison with commute data
- Transparent AI with source attribution
- Responsible AI practices documented

Faithfulness: 0.77 | Answer Relevancy: 1.00 | 4 Phases Planned | 3 LLM Providers | 48 Dependencies

Thank You — Questions?

Ajai | ajaikm@hotmail.com | March 2026